

1. Data Structures and Algorithms

A data structure is a way of organizing and storing data in a computer so it can be used easily. An algorithm is a set of step-by-step instructions used to solve a problem or perform a task. For example, an array can be used to store student grades, while an algorithm can be used to search for a specific grade in the array. Reference: MIT OpenCourseWare – Introduction to Algorithms.

2. Linear Data Structure

A linear data structure is a way of organizing data where the elements are arranged one after another in a sequence. Each element has a specific position in the sequence. Examples of linear data structures include arrays, linked lists, stacks, and queues. Reference: GeeksforGeeks – Introduction to Linear Data Structures.

3. One-Dimensional Array

A one-dimensional array is a collection of data stored in a single sequence or line. Each item in the array has an index, which is its position. For example, in $[10, 20, 30, 40]$, the value 10 is at index 0 and 30 is at index 2. Arrays are commonly used to store multiple values of the same type. Reference: GeeksforGeeks – Array Data Structure.

4. Big O Notation

Big O notation is used to describe how much time or resources an algorithm may need as the amount of data increases. It helps us understand how efficient an algorithm is. For example, $O(n)$ means the amount of work increases as the number of data items increases, while $O(\log n)$ means the algorithm can work more efficiently by reducing the amount of data it needs to check. Reference: MIT OpenCourseWare – Big O and Theta.

5. Basic Searching Algorithms

A searching algorithm is a method used to find a specific item or value from a collection of data. For example, if an array contains $[10, 20, 30, 40, 50]$ and we want to find 40, a searching algorithm helps locate that value. Two basic searching algorithms are Linear Search and Binary Search. Reference: OpenDSA – Searching in an Array.

6. Linear Search

Linear Search is a simple searching method that checks each item one by one, starting from the first item, until the desired value is found or the end of the data is reached. For example, to find 40 in $[10, 20, 30, 40]$, it checks 10, then 20, then 30, and finally

40. Linear search does not require the data to be sorted, and its worst-case time complexity is $O(n)$. Reference: OpenDSA – Searching in an Array.

7. Binary Search

Binary Search is a searching method that finds an item by repeatedly dividing the search area into half. It requires the data to be sorted first. For example, when searching for 60 in $[10, 20, 30, 40, 50, 60, 70]$, it first checks the middle value and then decides whether to search the left or right half. Its worst-case time complexity is $O(\log n)$, making it efficient for large, sorted collections. Reference: OpenDSA – Binary Search.